

Ime in priimek:

Datum:

# Načrtovanje in upravljanje informacijsko-komunikacijskih sistemov (NUKS)

## Laboratorijska vaja 7: Funkcije kot storitev

### Namen vaje

- Spoznavanje z funkcijami kot storitvami
- Prepoznavanje lastnosti, prednosti in slabosti takšnih sistemov proti monolitnim

Razvoj monolitnih aplikacij je dobro razumljiv in podprt v vseh danes prisotnih razvojnih okoljih. Celotna poslovna logika aplikacije je zbrana na enem mestu ter razdeljena po modulih, ki komunicirajo prek programskih klicev. Aplikacija vključuje tudi vse potrebne odvisnosti za delovanje (knjižnice, razširitve...). Prenos in namestitve teh storitev na strežniške sisteme je enostaven postopek, rešitev v obliki izvršljivih datotek ali s kopiranjem direktorijske strukture prenesemo v produkcijsko izvajalno okolje. Različni sestavni deli aplikacije so med seboj močno sklopljeni (medprocesna komunikacija), za uveljavitev sprememb enega dela sistema je potrebno celotno aplikacijo ponovno namestiti. Življenjski cikel takih aplikacij je po navadi dolg, sklopljenost sistema namreč ne omogoča enostavne menjave sestavnih delov ali celo programskega okolja.

Arhitektura mikrorstitev rešuje nekatere probleme monolitnih aplikacij, na račun večje kompleksnosti celotnega sistema. Šibka sklopljenost sestavnih delov je glavna prednost arhitekture, komunikacija med njimi pa običajno poteka preko lahkih omrežnih tehnologij (HTTP) ali asinhronih sporočilnih sistemov (sporočilne vrste, dogodkovno gnani sistemi). Arhitektura nam omogoča enostavnejšo menjavo sestavnih delov aplikacije, kar privede do krajšega življenjskega cikla in uvedbe praks sprotne dostave in sprotne integracije.

Oba pristopa prinašata tako prednosti kot slabosti. Na primeru Openfaas boste spoznali in preučili sistem funkcij kot storitev (»Function as a service«). Funkcije kot storitev predstavljajo najmanjšo enoto, do katere lahko razdružimo sistem.

Pri vajah boste uporabljali virtualke, podatke za dostop (**IP**, **uporabniško ime** in **geslo**) boste dobili od asistenta. Uporabljajte lahko ukazno vrstico (Putty, terminal ali alternativa). Za lažje delo pa bomo uporabljali tudi **Visual Studio Code**, ki ima podporo za Remote Explorer in še posebej bo dobrodošla razširitev Docker.

## 1. Registracija na portalu Docker Hub (<https://hub.docker.com/>)

Uporabniško ime: \_\_\_\_\_

\* zapomnite si tudi geslo, saj ga boste potrebovali na prihodnjih vajah

## 2. Na virtualki namestite Docker.

Sledite navodilom (<https://docs.docker.com/engine/install/ubuntu/#installation-methods>):

```
# ----- INSTALL DOCKER -----  
  
sudo apt-get update  
  
sudo apt-get -y install \  
apt-transport-https \  
ca-certificates \  
curl \  
gnupg-agent \  
software-properties-common  
  
sudo mkdir -m 0755 -p /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
  
echo "deb [arch=$(dpkg --print-architecture)] signed-by=/etc/apt/keyrings/docker.gpg\  
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" |  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
  
sudo apt-get update  
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin  
  
sudo systemctl enable docker  
  
# ----- INSTALL DOCKER-COMPOSE -----  
printf '\n\nInstalling docker-compose: \n'  
  
sudo curl -L "https://github.com/docker/compose/releases/download/1.27.4/docker-compose-$(uname -s)-  
$(uname -m)" -o /usr/local/bin/docker-compose
```

### 3. Inštalacija Openfaas z opcijo Openfaasd

```
# ----- INSTALL OpenFaas -----  
  
cd /tmp  
git clone https://github.com/openfaas/faasd  
cd faasd  
sudo ./hack/install.sh  
  
# ----- CHECK -----  
  
sudo systemctl status faasd  
sudo systemctl status faasd-provider
```

#### a. Preko UI vmesnika preverite uspešnost namestitve in postavite svojo prvo funkcijo

UI openfaas vmesnik najdete na vratih 8080

Url UI vmesnika: \_\_\_\_\_

Uporabniško ime: admin

Geslo: \_\_\_\_\_

Izpis funkcije NodeInfo:

- b. Na portalu docker.hub poiščite generator ikon (identicon). Funkcijo deploy-ajte preko terminala in preverite izpis za vaše ime ali priimek.**

Url repozitorija: \_\_\_\_\_

Ukaz za deploy: \_\_\_\_\_

Rezultat funkcije:

Vhod: \_\_\_\_\_

Rezultat:

- c. Spišite funkcijo za seštevanje števil.**

Pomagajte si s predlogami.

```
# ----- PRENOS PREDLOG -----  
faas-cli template pull  
  
# ----- LISTA PREDLOG -----  
faas-cli template store list  
  
# ----- EXAMPLE -----  
faas-cli new sumfunction --lang python3  
faas-cli build -f sumfunction.yml  
sudo faas-cli push -f sumfunction.yml  
faas-cli deploy --image username/functionname --name localfunctionname
```

d. (DODATEK) Zaradi preobremenjenosti strežnika, bi le tega radi razbremenili nekaterih nalog. Ena od nalog, ki ni odvisna od nobene druge naloge, je funkcija, ki sliko priredi na ustrezno višino. Spišite funkcijo za povečanje/pomanjšanje slike na 100px višine, pri tem pa ohranite razmerje dolžina/širina.

- <https://github.com/openfaas/classic-watchdog>
- Imagemagic: <https://imagemagick.org/index.php>

**Domača naloga do vaje (13.4.2023):**

- V svoj projekt vključite eno funkcijo kot storitev in jo implementirajte v okolju OpenFaas

**Komentar, zapiski vaje:**